KITCHEN MATE: RECIPE RECOMMENDATION SYSTEM USING PYTHON AND TKINTER

Submitted by

Parv Attrey - 26MCA20069

in partial fulfillment for the award of the degree of

Master of Computer Applications (MCA)

IN

Computer Applications



Chandigarh University

October - 2026

CASE STUDY QUESTIONS

1. Define the problem statement of your Python project and explain its objectives.

Problem Statement:

In daily life, people often stand in front of their kitchen shelves wondering what to cook with the few ingredients they have on hand. Searching the internet for recipes usually brings up complicated dishes that require extra grocery shopping, which wastes both time and money. Furthermore, many recipe apps require an active internet connection and are cluttered with ads. The purpose of this project is to build **Kitchen Mate**, a simple, offline desktop application that recommends Indian vegetarian recipes based on the exact ingredients a user already has at home. It also acts as a digital cookbook where users can browse existing dishes and save their own personal recipes.

Objectives:

- To build an easy-to-use recipe recommendation desktop application using Python.
- To design a clean, multi-tab graphical user interface (GUI) using Tkinter with separate sections for searching, browsing, and adding recipes.
- To automatically create and manage a local CSV database containing 50 pre-loaded Indian vegetarian recipes so the app works without an internet connection.
- To implement a smart ingredient-matching algorithm that calculates how well a recipe matches the user's available ingredients and ranks the best options at the top.
- To allow users to browse all recipes alphabetically and permanently add their own custom recipes to the CSV file.
- To display helpful pop-up alerts and warnings when input fields are left blank or when a new recipe is successfully saved.

2. Explain the selected algorithm with pseudocode or a flowchart.

This project uses a **Substring Ingredient-Matching and Scoring Algorithm**. Instead of requiring the user to type the exact phrase used in the recipe (for example, typing *"ripe tomato"* or *"paneer cubes"*), the algorithm cleans the text, converts everything to

lowercase, and checks if the user's ingredient word appears anywhere inside the recipe's required ingredient name.

For every recipe that has at least one matching ingredient, the program calculates a match percentage score using the formula:

$$\text{Match Score} = \left(\frac{\text{Number of Matched Ingredients}}{\text{Total Required Ingredients in Recipe}}\right) \times 100$$

Finally, the recipes are sorted from highest score to lowest score so the most convenient dishes appear first.

Pseudocode:

1. **START**
2. Check if kitchen_mate_recipes.csv exists on the computer; if not, create it and write the 50 default Indian vegetarian recipes into it.
3. Load all recipes from the CSV file into a Python list of dictionaries.
4. Display the main Tkinter window with three tabs: **Smart Chef**, **Recipe Book**, and **Add Recipe**.
5. **In the Smart Chef Tab:** When the user enters comma-separated ingredients and clicks *Find Recipes*, read the input.
6. If the input box is empty or still has the default placeholder text, show a warning message.
7. Otherwise, split the user's input by commas, remove extra spaces, and convert all words to lowercase.
8. Loop through each recipe in the list and split its required ingredients by commas.
9. For each required ingredient in the recipe, check if any user ingredient is a substring of that required ingredient; if yes, increase matched_count by 1.
10. If matched_count is greater than 0, calculate the percentage score: (matched_count / total_required_ingredients) * 100 and add the recipe to the matches list.
11. Sort the matching recipes in descending order based on their score and display them in the results list.

12. When the user clicks a recipe in the list, display its full title, bulleted ingredients, and step-by-step instructions on the right side.
13. **In the Recipe Book Tab:** Sort all recipes alphabetically by name and let the user click any dish to view its recipe.
14. **In the Add Recipe Tab:** Read the new recipe name, ingredients, and instructions; if all fields are filled, append the new recipe to the CSV file and refresh the app's data.
15. **END** when the user closes the application window.

Flowchart (text form):

Start → Check & Load CSV Recipe File → Open GUI Window → Go to Smart Chef Tab → Enter Ingredients & Click *Find Recipes* → Is Input Valid? → **No:** Show Warning Pop-up → **Yes:** Compare User Ingredients with Each Recipe Using Substring Check → Calculate Match Score (%) → Sort Recipes by Highest Score → Display Matched List → Click a Recipe to View Full Instructions → Close Application

3. Implement the solution using Python Language.

The following code uses Python's built-in tkinter, csv, and os modules to build the interface and manage the recipe dataset. Save it in a file named Pythonproject.py and run it using Python.

```
import tkinter as tk
from tkinter import ttk, messagebox, scrolledtext
import csv
import os

# --- APP CONFIGURATION & COLORS ---
APP_TITLE = "Kitchen Mate □🍳 - Your Recipe Recommender"
FILE_NAME = "kitchen_mate_recipes.csv"
BG_COLOR = "#FFFDF7"       # Creamy White
PRIMARY_COLOR = "#E25822"  # Flame Orange/Saffron
SEC_COLOR = "#4CAF50"      # Fresh Green
TEXT_COLOR = "#2B2B2B"     # Dark Grey
FONT_TITLE = ("Helvetica", 18, "bold")
FONT_HEADING = ("Helvetica", 14, "bold")
FONT_BODY = ("Helvetica", 11)
```

```
    # --- INDIAN VEG RECIPES DATASET (Sample recipes shown; 50 recipes in full CSV) ---
DEFAULT_RECIPES = [
    ("Aloo Gobi", "Potato, Cauliflower, Red Tomato, Onion, Cumin, Turmeric, Salt",
     "1. Heat oil in a pan.\n2. Add cumin seeds and chopped onions.\n3. Add chopped red tomatoes and cook until soft.\n4. Add potatoes and cauliflower.\n5. Add spices, cover, and cook until tender.\n6. Garnish with coriander."),

    ("Palak Paneer", "Spinach, Paneer cubes, Onion, Garlic, Green Chili, Garam Masala, Cream",
     "1. Blanch spinach and puree it.\n2. Sauté onion, garlic, and green chili.\n3. Add spinach puree and spices. Cook for 5 mins.\n4. Add paneer cubes and simmer.\n5. Finish with a swirl of cream."),

    ("Chana Masala", "Chickpeas, Onion, Ripe Tomato, Ginger, Garlic, Chole Masala",
     "1. Soak and boil chickpeas.\n2. Sauté ginger, garlic, and onions.\n3. Add pureed ripe tomato and chole masala.\n4. Cook until oil separates.\n5. Add chickpeas, simmer for 10 mins."),

    ("Dal Makhani", "Black Urad Dal, Kidney Beans, Butter, Tomato Puree, Cream, Garlic",
     "1. Soak and boil dal and beans until soft.\n2. In a pan, add butter and garlic paste.\n3. Add tomato puree and cook well.\n4. Mix in the boiled dal and simmer for 30 mins.\n5. Add cream and serve hot."),

    ("Paneer Butter Masala", "Paneer, Butter, Tomato, Cashew, Honey, Kasuri Methi",
     "1. Boil tomatoes and cashews, then puree.\n2. Melt butter, add the puree and cook.\n3. Add spices and a touch of honey.\n4. Add paneer cubes and crushed Kasuri Methi.\n5. Simmer for 2 minutes.")
    # ... [Remaining 45 Indian Vegetarian Recipes follow the exact same tuple format: (Name, Ingredients, Instructions)] ...
]

def initialize_csv():
    """Creates the CSV file with 50 recipes if it doesn't exist."""
    if not os.path.exists(FILE_NAME):
        with open(FILE_NAME, mode='w', newline='', encoding='utf-8') as file:
            writer = csv.writer(file)
            writer.writerow(["Recipe_Name", "Ingredients", "Instructions"])
```

```
            for recipe in DEFAULT_RECIPES:
                writer.writerow(recipe)
class KitchenMateApp:
    def __init__(self, root):
        self.root = root
        self.root.title(APP_TITLE)
        self.root.geometry("800x650")
        self.root.configure(bg=BG_COLOR)
        self.recipes = [] # List to hold dictionaries of recipes
        initialize_csv()
        self.load_data()
        # Styling
        style = ttk.Style()
        style.theme_use('clam')
        style.configure("TNotebook", background=BG_COLOR, borderwidth=0)
        style.configure("TNotebook.Tab", font=FONT_BODY, padding=[15, 5],
background="#E0E0E0")
        style.map("TNotebook.Tab", background=[("selected",
PRIMARY_COLOR)], foreground=[("selected", "white")])
        style.configure("TButton", font=FONT_BODY, background=SEC_COLOR,
foreground="white", padding=6)
        style.map("TButton", background=[("active", "#45a049")])
        # Title Label
        title_lbl = tk.Label(self.root, text=APP_TITLE, font=FONT_TITLE,
bg=BG_COLOR, fg=PRIMARY_COLOR, pady=10)
        title_lbl.pack()
        self.notebook = ttk.Notebook(self.root)
        self.notebook.pack(expand=True, fill="both", padx=20, pady=10)
        self.tab_find = tk.Frame(self.notebook, bg=BG_COLOR)
        self.tab_browse = tk.Frame(self.notebook, bg=BG_COLOR)
        self.tab_add = tk.Frame(self.notebook, bg=BG_COLOR)
        self.notebook.add(self.tab_find, text=" 🔍 Smart Chef ")
        self.notebook.add(self.tab_browse, text=" 📖 Recipe Book ")
        self.notebook.add(self.tab_add, text=" ➕ Add Recipe ")
        self.setup_find_tab()
        self.setup_browse_tab()
        self.setup_add_tab()
    def load_data(self):
        """Loads recipes from the CSV file."""
        self.recipes.clear()
        try:
            with open(FILE_NAME, mode='r', encoding='utf-8') as file:
                reader = csv.DictReader(file)
                for row in reader:
                    self.recipes.append(row)
```

```
        except Exception as e:
            messagebox.showerror("Error", f"Could not load recipes: {e}")
    # ================ TAB 1: SMART CHEF (ALGORITHM) ================
    def setup_find_tab(self):
        lbl = tk.Label(self.tab_find, text="What ingredients do you have?
(Comma separated)", font=FONT_HEADING, bg=BG_COLOR, fg=TEXT_COLOR)
        lbl.pack(pady=(15, 5))
        self.inventory_entry = tk.Entry(self.tab_find, font=FONT_BODY,
width=50)
        self.inventory_entry.pack(pady=5)
        self.inventory_entry.insert(0, "e.g., tomato, paneer, onion")
        btn = ttk.Button(self.tab_find, text="Find Recipes",
command=self.find_recipes)
        btn.pack(pady=10)
        # PanedWindow to show Results List and Recipe Details side-by-side
        pane = ttk.PanedWindow(self.tab_find, orient=tk.HORIZONTAL)
        pane.pack(expand=True, fill="both", padx=10, pady=10)
        list_frame = tk.Frame(pane, bg=BG_COLOR)
        self.result_listbox = tk.Listbox(list_frame, font=FONT_BODY,
selectbackground=PRIMARY_COLOR)
        self.result_listbox.pack(expand=True, fill="both", side=tk.LEFT)
        scrollbar1 = ttk.Scrollbar(list_frame, orient="vertical",
command=self.result_listbox.yview)
        scrollbar1.pack(side=tk.RIGHT, fill="y")
        self.result_listbox.config(yscrollcommand=scrollbar1.set)
      self.result_listbox.bind('<<ListboxSelect>>', self.on_result_select)
pane.add(list_frame, weight=1)
        self.result_detail = scrolledtext.ScrolledText(pane,
font=FONT_BODY, wrap=tk.WORD, bg="#FFF8F0")
        pane.add(self.result_detail, weight=2)
    def find_recipes(self):
        inventory_input = self.inventory_entry.get().strip()
        if not inventory_input or inventory_input == "e.g., tomato,
paneer, onion":
            messagebox.showwarning("Input Error", "Please enter some
ingredients.")
            return
        # Clean up inventory input
        inventory = [item.strip().lower() for item in
inventory_input.split(",") if item.strip()]
        matches = []
        for recipe in self.recipes:
            # Clean up recipe ingredients
            recipe_ingredients = [ing.strip().lower() for ing in
recipe['Ingredients'].split(",")]
```

```
            # ALGORITHM
            matched_count = 0
            for req_ing in recipe_ingredients:
                for inv_item in inventory:
                    if inv_item in req_ing:
                        matched_count += 1
                        break
            # Calculate a match score
            total_req = len(recipe_ingredients)
            if matched_count > 0:
                score = (matched_count / total_req) * 100
                matches.append({"recipe": recipe, "score": score,
"matched": matched_count, "total": total_req})
        # Sort by best match (highest score first)
        matches.sort(key=lambda x: x['score'], reverse=True)
        # Update Listbox
        self.result_listbox.delete(0, tk.END)
        self.current_matched_recipes = [] # Keep a reference list aligned
with listbox
        if not matches:
            self.result_listbox.insert(tk.END, "No matching recipes found.
:(") self.result_detail.delete(1.0, tk.END)
            return
        for match in matches:
            r_name = match['recipe']['Recipe_Name']
 display_text = f"{r_name} ({match['matched']}/{match['total']} match)"
 self.result_listbox.insert(tk.END, display_text)
            self.current_matched_recipes.append(match['recipe'])
    def on_result_select(self, event):
        selection = self.result_listbox.curselection()
        if selection:
            index = selection[0]
            if index < len(self.current_matched_recipes):
                recipe = self.current_matched_recipes[index]
                self.show_recipe_details(recipe, self.result_detail)
    # ================ TAB 2: RECIPE BOOK (BROWSE) ================
    def setup_browse_tab(self):
        pane = ttk.PanedWindow(self.tab_browse, orient=tk.HORIZONTAL)
        pane.pack(expand=True, fill="both", padx=10, pady=10)
        # Browse Listbox
        list_frame = tk.Frame(pane, bg=BG_COLOR)
        self.browse_listbox = tk.Listbox(list_frame, font=FONT_BODY,
selectbackground=PRIMARY_COLOR)
        self.browse_listbox.pack(expand=True, fill="both", side=tk.LEFT)
```

```
        scrollbar2 = ttk.Scrollbar(list_frame, orient="vertical",
command=self.browse_listbox.yview)
        scrollbar2.pack(side=tk.RIGHT, fill="y")
        self.browse_listbox.config(yscrollcommand=scrollbar2.set)
        self.browse_listbox.bind('<<ListboxSelect>>',
self.on_browse_select)
        pane.add(list_frame, weight=1)
        self.browse_detail = scrolledtext.ScrolledText(pane,
font=FONT_BODY, wrap=tk.WORD, bg="#FFF8F0")
        pane.add(self.browse_detail, weight=2)
        self.populate_browse_list()
    def populate_browse_list(self):
        self.browse_listbox.delete(0, tk.END)
        sorted_recipes = sorted(self.recipes, key=lambda x:
x['Recipe_Name'])
        self.current_browse_recipes = sorted_recipes
        for recipe in sorted_recipes:
            self.browse_listbox.insert(tk.END, recipe['Recipe_Name'])
    def on_browse_select(self, event):
        selection = self.browse_listbox.curselection()
        if selection:
            index = selection[0]
            recipe = self.current_browse_recipes[index]
            self.show_recipe_details(recipe, self.browse_detail)
    # ================= TAB 3: ADD RECIPE =================
    def setup_add_tab(self):
 form_frame = tk.Frame(self.tab_add, bg=BG_COLOR)
        form_frame.pack(pady=20, padx=50, fill="both", expand=True)
        tk.Label(form_frame, text="Recipe Name:", font=FONT_HEADING,
bg=BG_COLOR).grid(row=0, column=0, sticky="w", pady=10)
        self.add_name = tk.Entry(form_frame, font=FONT_BODY, width=40)
        self.add_name.grid(row=0, column=1, pady=10, padx=10)
        tk.Label(form_frame, text="Ingredients:\n(Comma separated)",
font=FONT_HEADING, bg=BG_COLOR).grid(row=1, column=0, sticky="nw",
pady=10)
        self.add_ingredients = tk.Text(form_frame, font=FONT_BODY,
width=40, height=4)
        self.add_ingredients.grid(row=1, column=1, pady=10, padx=10)
        tk.Label(form_frame, text="Instructions:\n(Step-by-step)",
font=FONT_HEADING, bg=BG_COLOR).grid(row=2, column=0, sticky="nw",
pady=10)
        self.add_instructions = tk.Text(form_frame, font=FONT_BODY,
width=40, height=8)
        self.add_instructions.grid(row=2, column=1, pady=10, padx=10)
```

```
        save_btn = ttk.Button(form_frame, text="Save Recipe to CSV",
command=self.save_recipe)
        save_btn.grid(row=3, column=0, columnspan=2, pady=20)
    def save_recipe(self):
        name = self.add_name.get().strip()
        ingredients = self.add_ingredients.get("1.0", tk.END).strip()
        instructions = self.add_instructions.get("1.0", tk.END).strip()
        if not name or not ingredients or not instructions:
            messagebox.showwarning("Incomplete", "Please fill out all
fields.") return
        try:
            with open(FILE_NAME, mode='a', newline='', encoding='utf-8')
as file:
                writer = csv.writer(file)
                writer.writerow([name, ingredients, instructions])
            messagebox.showinfo("Success", f"'{name}' has been added!")
            # Clear form
            self.add_name.delete(0, tk.END)
            self.add_ingredients.delete("1.0", tk.END)
            self.add_instructions.delete("1.0", tk.END)
            self.load_data()
            self.populate_browse_list()
        except Exception as e:
            messagebox.showerror("Error", f"Failed to save recipe: {e}")
    # ================= HELPER FUNCTIONS =================
    def show_recipe_details(self, recipe, text_widget):
        """Formats and displays recipe data inside the given ScrolledText
widget."""
        text_widget.config(state=tk.NORMAL)
        text_widget.delete(1.0, tk.END)
        # Display Name
        text_widget.insert(tk.END, f"{recipe['Recipe_Name']}\n", "title")
        text_widget.insert(tk.END, "=" * 40 + "\n\n")
        # Display Ingredients
        text_widget.insert(tk.END, "🛒 Ingredients:\n", "heading")
        ingredients_list = recipe['Ingredients'].split(',')
        for ing in ingredients_list:
            text_widget.insert(tk.END, f" • {ing.strip()}\n")
        text_widget.insert(tk.END, "\n")
        # Display Instructions
        text_widget.insert(tk.END, "🍳 Instructions:\n", "heading")
        text_widget.insert(tk.END, f"{recipe['Instructions']}\n")
        # Apply Tags for colors/fonts
        text_widget.tag_configure("title", font=("Helvetica", 16, "bold"),
foreground=PRIMARY_COLOR)
```

```
        text_widget.tag_configure("heading", font=("Helvetica", 12,
"bold"), foreground=SEC_COLOR)

        text_widget.config(state=tk.DISABLED)
if __name__ == "__main__":
    root = tk.Tk()
    app = KitchenMateApp(root)
    root.mainloop()
```

4. Explain the Python libraries, functions, tools used in your project and justify their selection.

Python:

Python is the core programming language used to build this application. It was chosen because its syntax is clean and easy to read.

Tkinter (tk):

Tkinter is Python's standard GUI library. It is used to build the main application window, frames, text labels, single-line input boxes (Entry), multi-line input boxes (Text), and the clickable Listbox for displaying recipe names. It was selected because it comes bundled with Python and does not require installing heavy third-party frameworks.

ttk (Themed Tkinter):

The ttk module provides modern-looking widgets and styling options. In this project, ttk.Notebook creates the three navigation tabs (*Smart Chef*, *Recipe Book*, and *Add Recipe*), ttk.PanedWindow creates the split-screen view between the recipe list and recipe details, and ttk.Style applies custom saffron and green colors.

messagebox:

The messagebox module is used to display pop-up dialog boxes. It shows warnings (showwarning) when a user forgets to enter ingredients or leaves form fields blank, confirmation alerts (showinfo) when a new recipe is saved, and error alerts (showerror) if a file issue occurs.

scrolledtext:

The scrolledtext module provides the ScrolledText widget, which adds a built-in vertical scrollbar to the recipe details area. This ensures long ingredient lists and multi-step cooking instructions remain easy to read without overflowing the screen.

csv and os Libraries:

The os module checks whether kitchen_mate_recipes.csv already exists on the computer using os.path.exists(). The csv module (csv.writer and csv.DictReader) reads and writes recipe records in a lightweight spreadsheet format

Lists, Tuples, and Dictionaries:

The 50 default recipes are stored as tuples inside the DEFAULT_RECIPES list. When loaded from the CSV file, each recipe is stored as a dictionary containing keys for Recipe_Name, Ingredients, and Instructions.

Class and Functions:

- initialize_csv() checks for the CSV file on startup and populates it with the 50 default recipes if the file is missing.
- KitchenMateApp is the main class that organizes all GUI tabs and application logic in an object-oriented way.
- load_data() reads the CSV file and loads all recipes into memory.
- find_recipes() runs the substring-matching algorithm, calculates match scores, and displays ranked results.
- populate_browse_list() sorts all recipes alphabetically for the *Recipe Book* tab.
- save_recipe() validates user input in the *Add Recipe* tab and appends the new dish to the CSV file.
- show_recipe_details() formats and displays the selected recipe's title, ingredients, and instructions, and locks the text box (state=tk.DISABLED) so users cannot accidentally edit the text while reading.

Development Tool:

Visual Studio Code (VS Code) or Python IDLE can be used to write, test, and run the script. The project relies strictly on Python's standard library and runs completely offline.

5. Test your Python project with different inputs and explain the results.

The following test cases verify the main features and error-handling of the Kitchen Mate application:

Test	Input / Action	Expected Result	Explanation
1	Enter tomato, paneer, onion in the *Smart Chef* tab and click Find Recipes	Matching paneer and tomato recipes appear in the list	The algorithm matches the ingredients, calculates the score, and ranks dishes like *Kadai Paneer (3/5 match)* and *Matar Paneer (3/6 match)* at the top.
2	Click on a recipe name (e.g., *Kadai Paneer*) in the search results list	Full recipe details appear in the right-hand panel	The <<ListboxSelect>> event triggers show_recipe_details() and displays the formatted title, ingredients, and cooking steps.
3	Leave the default placeholder text (e.g., tomato, paneer, onion) or an empty box and click Find Recipes	Warning pop-up appears saying *"Please enter some ingredients."*	The input validation check prevents the search algorithm from running without valid user ingredients.
4	Enter an uncommon item like chocolate and click Find Recipes	The listbox displays *"No matching recipes found. :("*	None of the recipes contain chocolate, so matched_count stays 0 and the details panel is cleared.
5	Switch to the Recipe Book tab and select *Aloo Gobi*	All 50+ recipes appear in alphabetical order, and *Aloo Gobi* details are shown	populate_browse_list() sorts the loaded recipes alphabetically from A to Z for easy reading.

6	Go to Add Recipe, fill in Name, Ingredients, and Instructions, and click Save Recipe to CSV	Success message appears and the form clears	The new recipe is appended to kitchen_mate_recipes.csv and immediately becomes searchable in the other tabs.
7	Go to Add Recipe, leave the Instructions box empty, and click Save Recipe to CSV	Warning pop-up appears saying *"Please fill out all fields."*	The save_recipe() function checks that all three fields contain text before writing to the CSV file.

Conclusion

The **Kitchen Mate Recipe Recommendation System** shows how Python and Tkinter can be combined with CSV file handling to solve an everyday household problem. By storing 50 classic Indian vegetarian recipes and using a flexible substring-matching algorithm, the app helps users quickly discover what dishes they can cook with the ingredients already in their kitchen. The project makes practical use of core Python concepts including object-oriented programming, lists, dictionaries, string methods, loops, and file I/O. In the future, this application can be expanded by adding recipe photos, cooking timers, dietary filters (such as vegan or high-protein), and an option to edit or delete saved recipes.